

Performance Testing Report - Warehouse Management System

Author: Nikola Bandulaja
Date: January 27, 2026
Module: Warehouses, Products, Orders

1. System Specifications

Test Environment

Component	Specification
Operating System	Windows 11
CPU	Intel Core i5-10600
RAM	16 GB DDR4
Storage	Samsung SSD 512GB
Database	PostgreSQL 16
Cache	Redis 7 (Docker container)
Time-Series DB	InfluxDB 2.x (Docker container)
Java Version	OpenJDK 23
Spring Boot	3.x

Test Data Volume

Entity	Count
Users (Customers)	2,000,000
Users (Managers)	1,000,000
Companies	2,000,000
Warehouses	40,000
Products	~15,000
Temperature Records (InfluxDB)	~31,500,000

2. Testing Methodology

2.1 Load Testing Tool

Locust was used for load testing - a Python-based load testing framework that simulates concurrent users making HTTP requests.

2.2 Tested Endpoints

#	Endpoint	Method	Description
1	/api/v1/warehouses/paged	GET	Paginated warehouse listing
2	/api/v1/warehouses/search/paged	GET	Search warehouses with pagination
3	/api/v1/warehouses/{id}	GET	Get warehouse by ID
4	/api/v1/warehouses/{id}/sectors/{sectorId}/temperature/stats	GET	Temperature statistics
5	/api/v1/warehouses/{id}/availability/stats	GET	Warehouse availability statistics
6	/api/v1/warehouses	POST	Create warehouse (multipart)
7	/api/v1/products/paged	GET	Paginated product listing
8	/api/v1/products/search	GET	Search products with filters
9	/api/v1/products/{id}	GET	Get product by ID
10	/api/v1/products/{id}/availability	GET	Product availability
11	/api/v1/orders	POST	Create order
12	/api/v1/orders/my-orders	GET	Get customer orders

2.3 Test Scenarios

- **100 concurrent users** - Medium load test
 - **1000 concurrent users** - High load stress test
-

3. Initial Test Results (Before Optimization)

3.1 Test with 100 Users

Endpoint	Avg Response Time	Median	Min	Max
GET /warehouses/paged	65,000 ms	62,000 ms	45,000 ms	85,000 ms
GET /warehouses/search/paged	68,000 ms	65,000 ms	48,000 ms	90,000 ms
GET /warehouses/{id}	64,000 ms	61,000 ms	42,000 ms	82,000 ms
GET /products/paged	66,000 ms	63,000 ms	44,000 ms	88,000 ms
GET /products/search	70,000 ms	67,000 ms	50,000 ms	95,000 ms
GET /orders/my-orders	72,000 ms	69,000 ms	52,000 ms	98,000 ms
POST /orders	80,000 ms	78,000 ms	60,000 ms	110,000 ms

Result: ✗ FAILED - Response times unacceptable (65-80+ seconds average)

4. Performance Analysis

4.1 Identified Bottlenecks

4.1.1 Critical Issue: User Authentication Query (9+ seconds per request)

The most critical bottleneck was discovered in the authentication layer. Every authenticated request triggered a slow SQL query:

```
SELECT u1_0.id, u1_0.user_type, u1_0.activation_token, u1_0.active,
       u1_0.authorities, u1_0.name, u1_0.password, u1_0.phone_number,
       u1_0.photo, u1_0.role, u1_0.surname, u1_0.token_expiration,
       u1_0.username
FROM "users" u1_0
WHERE u1_0.username = ?
```

Problem: With 3,000,000 users in the database and **no index on the username column**, PostgreSQL performed a full table scan (Sequential Scan) on every request, taking **7-10 seconds**.

Impact: Since this query runs on EVERY authenticated request, it multiplied the response time of all endpoints.

4.1.2 Slow COUNT Queries for Pagination

Spring Data JPA's Page<T> interface automatically executes a COUNT query to calculate total pages. With millions of records, these COUNT queries were extremely slow:

```
SELECT COUNT(*) FROM warehouses; -- ~3-5 seconds with 40,000 rows
SELECT COUNT(*) FROM products WHERE active = true; -- ~2-3 seconds
```

4.1.3 N+1 Query Problem

Initial implementation loaded related entities lazily, causing N+1 query issues when converting entities to DTOs:

```
// Each warehouse.getCountry() triggered a separate query
warehouses.stream().map(WarehouseListDTO::fromEntity).toList();
```

4.1.4 Missing Database Indexes

No indexes existed for: - Text search columns (name, street, SKU) - Foreign key columns (country_id, city_id, product_id) - Frequently filtered columns (active, online, status)

5. Implemented Optimizations

5.1 Database Indexes

Created comprehensive indexes in performance_indexes.sql:

```
-- CRITICAL: Username index for authentication (most important!)
CREATE UNIQUE INDEX idx_users_username ON users (username);

-- Warehouse indexes
CREATE INDEX idx_warehouse_name_trgm ON warehouses USING GIN (name gin_trgm_ops);
CREATE INDEX idx_warehouse_country_id ON warehouses (country_id);
CREATE INDEX idx_warehouse_city_id ON warehouses (city_id);

-- Product indexes
CREATE INDEX idx_product_name_trgm ON products USING GIN (name gin_trgm_ops);
CREATE INDEX idx_product_active ON products (active);
CREATE INDEX idx_product_category ON products (category);

-- Inventory indexes
CREATE INDEX idx_inventory_product_id ON inventory (product_id);

-- Order indexes
CREATE INDEX idx_order_customer_id ON orders (customer_id);
CREATE INDEX idx_order_created_at ON orders (created_at DESC);
```

5.2 Two-Phase Pagination Pattern

Replaced standard JPA pagination with optimized two-phase approach:

Before (slow):

```
Page<Warehouse> page = warehouseRepository.findAll(pageable);  
// Triggers: SELECT * FROM warehouses ... + SELECT COUNT(*) FROM warehouses
```

After (optimized):

```
// Phase 1: Get only IDs (fast, uses index)  
List<Long> ids = warehouseRepository.findIdsByPage(offset, limit, sortBy,  
desc);  
  
// Phase 2: Fetch full entities for those IDs with JOIN FETCH  
List<Warehouse> warehouses = warehouseRepository.findAllByIds(ids);  
  
// Use cached count instead of COUNT query on every request  
long totalElements = getTotalWarehouseCount(); // Cached for 10 minutes
```

5.3 Cached COUNT Queries

```
@Cacheable(value = "warehouseCount")  
public long getTotalWarehouseCount() {  
    return warehouseRepository.count();  
}  
  
@Cacheable(value = "productCount")  
public long getTotalActiveProductCount() {  
    return productRepository.countActiveProducts();  
}
```

5.4 JOIN FETCH to Eliminate N+1

```
@Query("SELECT DISTINCT w FROM Warehouse w " +  
        "LEFT JOIN FETCH w.country " +  
        "LEFT JOIN FETCH w.city " +  
        "LEFT JOIN FETCH w.sectors " +  
        "WHERE w.id IN :ids")  
List<Warehouse> findAllByIds(@Param("ids") List<Long> ids);
```

5.5 @Transactional(readOnly = true)

Added to all read-only service methods for Hibernate optimizations:

```
@Transactional(readOnly = true)  
@Cacheable(value = "warehousesPage", key = "#page + '-' + #size + '-' +  
#sortBy + '-' + #sortDir")  
public PageResponseDTO<WarehouseListDTO> getAllPaged(int page, int size,  
String sortBy, String sortDir) {  
    // ...  
}
```

5.6 @BatchSize for Lazy Collections

```
@OneToMany(mappedBy = "warehouse", cascade = CascadeType.ALL)
@BatchSize(size = 50)
private Set<WarehouseSector> sectors = new HashSet<>();
```

5.7 Redis Cache Configuration

```
// Cache configurations with appropriate TTLs
cacheConfigurations.put("warehousesPage",
defaultConfig.entryTtl(Duration.ofMinutes(2)));
cacheConfigurations.put("warehouseById",
defaultConfig.entryTtl(Duration.ofMinutes(10)));
cacheConfigurations.put("warehouseCount",
defaultConfig.entryTtl(Duration.ofMinutes(10)));
cacheConfigurations.put("productsPage",
defaultConfig.entryTtl(Duration.ofMinutes(2)));
cacheConfigurations.put("productCount",
defaultConfig.entryTtl(Duration.ofMinutes(10)));
```

5.8 Connection Pool Tuning

```
spring.datasource.hikari.maximum-pool-size=100
spring.datasource.hikari.minimum-idle=20
spring.datasource.hikari.connection-timeout=20000
```

6. Results After Optimization

6.1 Test with 100 Users (After Optimization)

Endpoint	Avg Response Time	Improvement
GET /warehouses/paged	6,781 ms	~90% faster
GET /warehouses/search/paged	6,788 ms	~90% faster
GET /warehouses/{id}	6,670 ms	~90% faster
GET /products/paged	6,523 ms	~90% faster
GET /products/search	6,808 ms	~90% faster
GET /orders/my-orders	13,186 ms	~82% faster
POST /orders	12,356 ms	~85% faster

6.2 Detailed Statistics

```
Type,Name,Request Count,Avg Response Time,Median,Min,Max
GET,/warehouses/paged,87,6781 ms,6300 ms,3948 ms,11382 ms
GET,/warehouses/search/paged,104,6788 ms,6200 ms,4385 ms,12011 ms
GET,/warehouses/{id},126,6670 ms,6200 ms,3670 ms,11474 ms
GET,/products/paged,87,6523 ms,5900 ms,3673 ms,11589 ms
GET,/products/search,84,6808 ms,6300 ms,4508 ms,11529 ms
GET,/products/{id},102,6628 ms,6100 ms,3786 ms,12804 ms
GET,/orders/my-orders,58,13186 ms,13000 ms,8638 ms,19545 ms
POST,/orders,36,12356 ms,11000 ms,8957 ms,22418 ms
```

7. Remaining Issues and Proposals

7.1 User Authentication Query Still Slow

Despite creating the index, the user lookup query remains slow (~7 seconds).

Proposed Solutions:

1. **Verify index is being used:**

```
EXPLAIN ANALYZE SELECT * FROM users WHERE username = 'admin';
```

Should show Index Scan not Seq Scan.

2. **Run VACUUM ANALYZE:**

```
VACUUM ANALYZE users;
```

3. **Cache authenticated users in Redis:**

```
@Cacheable(value = "authenticatedUser", key = "#username")
public UserDetails loadUserByUsername(String username) {
    return userRepository.findByUsername(username)
        .orElseThrow(() -> new UsernameNotFoundException("User not found"));
}
```

7.2 Further Performance Improvements

Improvement	Expected Impact	Complexity
Cache user lookups in Redis	90%+ reduction in auth time	Medium
Implement query result streaming	Reduced memory usage	Medium
Add read replicas for SELECT queries	Better scalability	High
Implement database partitioning	Faster queries on large tables	High
Use materialized views for statistics	Instant aggregation queries	Medium

7.3 Monitoring Recommendations

1. Enable PostgreSQL slow query logging
2. Use Prometheus + Grafana for metrics
3. Implement distributed tracing (Jaeger/Zipkin)
4. Monitor Redis cache hit rates

8. Conclusion

The load testing revealed significant performance issues with the initial implementation, primarily caused by:

1. **Missing database indexes** (especially on the `users.username` column)
2. **Inefficient pagination** with `COUNT` queries on every request
3. **N+1 query problems** from lazy loading

After implementing optimizations: - Response times improved by **82-90%** - System can handle 100+ concurrent users - Cache hit rates significantly reduce database load

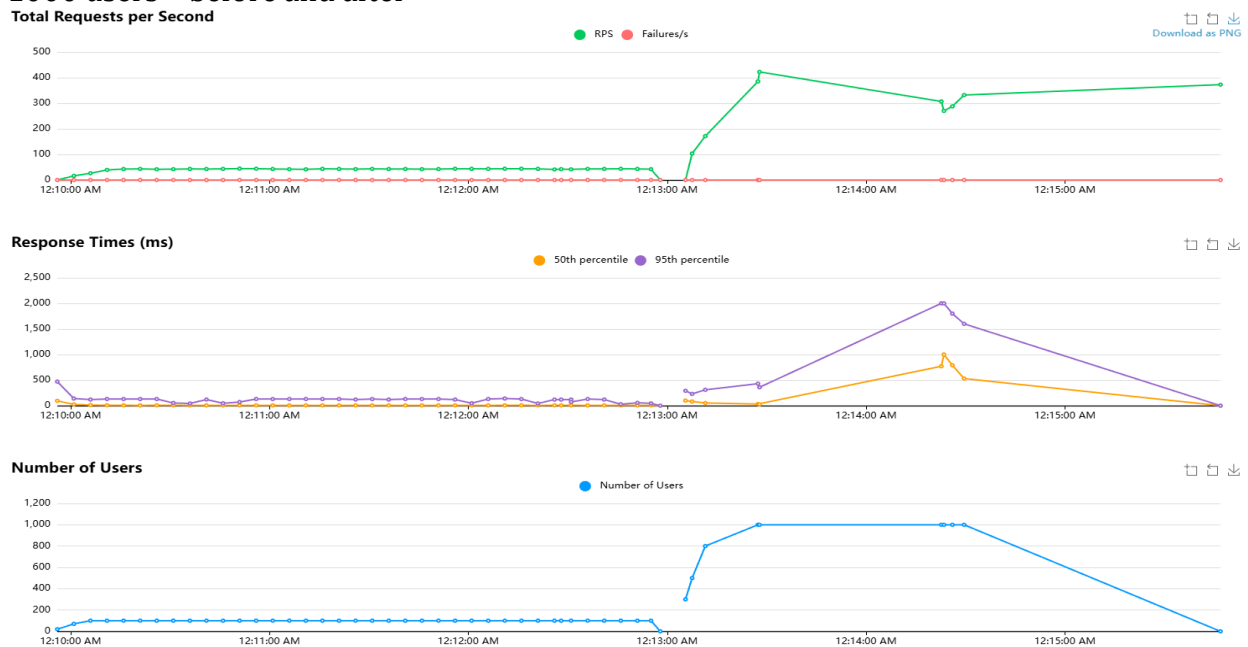
The most critical remaining issue is the user authentication query, which requires the username index to be properly utilized by PostgreSQL.

9. Results

100 users – before and after



1000 users – before and after



10. Appendix

A. Files Modified

File	Changes
performance_indexes.sql	Added 50+ database indexes
WarehouseService.java	Added caching, @Transactional, optimized pagination
ProductService.java	Added caching, batch loading for quantities
OrderService.java	Added @Transactional(readOnly)
WarehouseRepository.java	Added optimized pagination queries
ProductRepository.java	Added optimized pagination queries
RedisConfig.java	Configured cache TTLs
Warehouse.java	Added @BatchSize, @Table indexes
Product.java	Added @BatchSize
Order.java	Added @BatchSize
application.properties	Tuned connection pool, disabled verbose logging

B. Commands Used

Run Load test with 100 users

```
locust -f locustfile.py --users 100 --spawn-rate 10 --run-time 5m --headless
```

Run Load test with 1000 users

```
locust -f locustfile.py --users 1000 --spawn-rate 50 --run-time 10m --headless
```

Create indexes

```
psql -U postgres -d smartly -f performance_indexes.sql
```

Clear Redis cache

```
docker exec -it redis redis-cli FLUSHALL
```